

Example repository

 README  MIT license

Bulletproof frontend testing

This repository supports a series of talks and workshops focused on testing frontend applications the bulletproof way. Its aim is to provide testing suites, cases and patterns as well as rich explanations of their rationales.

It contains an example React SPA with a thin layer of supporting backend. Testing suites cover integration, functionality, accessibility, visual regression and so on.

The talks were performed during:

- [Voxxed Days Crete](#) conference talk (26-27.09.2025 - Heraklion, Crete Island, Greece)
- [React Alicante](#) workshop (02-04.10.2025 - Alicante, Spain)



<https://github.com/LukaszNowakPL/bulletproof-frontend-testing>

Component wrappers and other patterns for making your tests bulletproof

Łukasz Nowak, Voxxed Days Crete
Heraklion, 26 September 2025



Łukasz Nowak

Senior frontend dev



Łukasz Nowak



LukaszNowakPL

Why do we suck hard on tests?

- They sometimes bring no value
- We don't understand proper tests approaches ...
- ... and literature sometimes doesn't help much
- They are tightly coupled with the source code
- Sooner or later, they become flaky
- Because we tend to overcomplicate them



Scope of the presentation

- Development related tests, which means:
 - Direct checks of non-compiled code
 - Simple, component-related scenarios and quick answers
 - Vitest ecosystem & Testing Library family
- Playwright and Cypress are cool, but not for this presentation





Add airport

Name

IATA Code

Country

Regions

- ☐ Aegean Islands
- ☐ Corfu Island (Kerkyra)
- ☐ Ha Long Island
- ☐ Rhodes Island

- ☐ Balearic Islands
- ☒ Crete Island
- ☐ Ionian Islands
- ☐ Tenerife Island

- ☐ Canary Islands
- ☐ Dodecanese Islands
- ☐ Maiorca Island

Vaccination Notes



”

You need only four test cases
to check such a page completely.

Łukasz Nowak's promise, Voxxed Days Crete 2025



Mistakes we usually do

Add airport

Name

Airport name is required

IATA Code

Country

Regions

- ☐ Aegean Islands
- ☐ Corfu Island (Kerkyra)
- ☐ Ha Long Island
- ☐ Rhodes Island

- ☐ Balearic Islands
- ☒ Crete Island
- ☐ Ionian Islands
- ☐ Tenerife Island

- ☐ Canary Islands
- ☐ Dodecanese Islands
- ☐ Maiorca Island

Vaccination Notes

Submit

Test with false-positive outcome

```
it('does not display validation error by default' , async () => {  
  // When component is rendered  
  render(<AddAirportView />)  
  
  // Then the validation message is not displayed  
  expect(screen.queryByText('Airport name is required'))  
    .not.toBeInTheDocument();  
});
```


Test case assumed to be fixed

```
it('does not display validation error by default' , async () => {  
  // When component is rendered  
  render(<AddAirportView />)  
  
  // And data loader appeared only during data fetching  
  expect(screen.getByTestId('progress-bar')).toBeInTheDocument();  
  await waitForElementToBeRemoved(screen.queryByTestId('progress-bar'))  
  
  // Then the validation message is not displayed  
  expect(screen.queryByText('Airport name is required'))  
    .not.toBeInTheDocument();  
});
```



Sorry, there is some connectivity error

We were unable to fetch some of data necessary to display the page. You can [restart data fetching](#) manually.

If it won't help, try to restart the app.

Add airport

Name

Airport name is required

IATA Code

Country

Regions

- ☐ Aegean Islands
- ☐ Corfu Island (Kerkyra)
- ☐ Ha Long Island
- ☐ Rhodes Island

- ☐ Balearic Islands
- ☒ Crete Island
- ☐ Ionian Islands
- ☐ Tenerife Island

- ☐ Canary Islands
- ☐ Dodecanese Islands
- ☐ Maiorca Island

Vaccination Notes

Submit

Testing implementation details instead of functionality

```
// src/const/routes.ts
const ROUTES = {
  AIRPORTS: '/airports'
} as const
```

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
  const {history} = render(<AddAirportView />)

  // Form fulfillment...

  await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
  expect(history.location.pathname).toEqual(ROUTES.AIRPORTS)
});
```

How such a test can fire back?

```
// src/const/routes.ts
const ROUTES = {
  AIRPORTS: '/assports'
} as const
```

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
  const {history} = render(<AddAirportView />)

  // Form fulfillment...

  await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
  expect(history.location.pathname).toEqual(ROUTES.AIRPORTS)
});
```

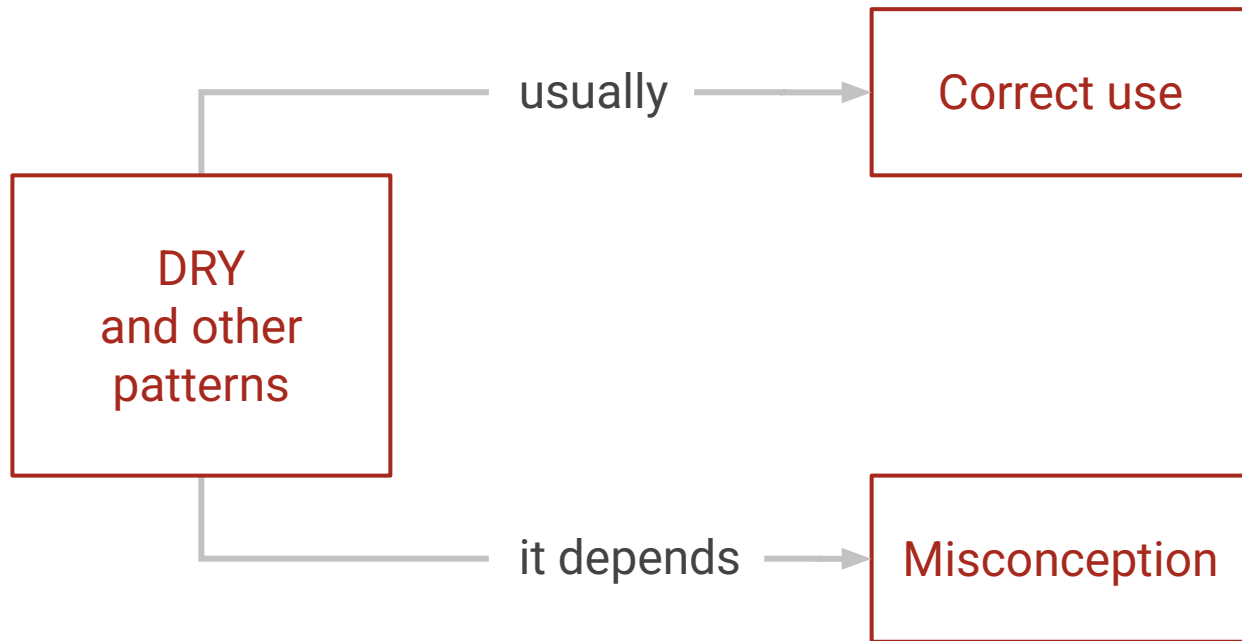
Testing functionality instead of implementation details

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
  const {history} = render(<AddAirportView />)

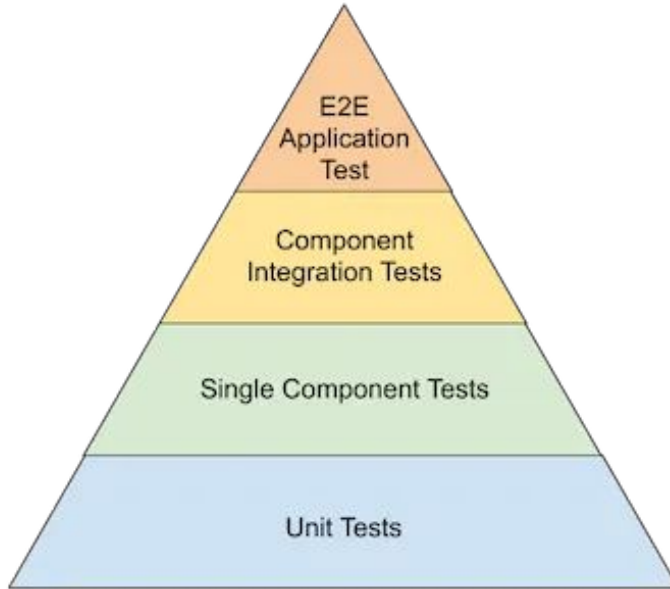
  // Form fulfillment...

  await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
  expect(history.location.pathname).toEqual('/airports')
});
```

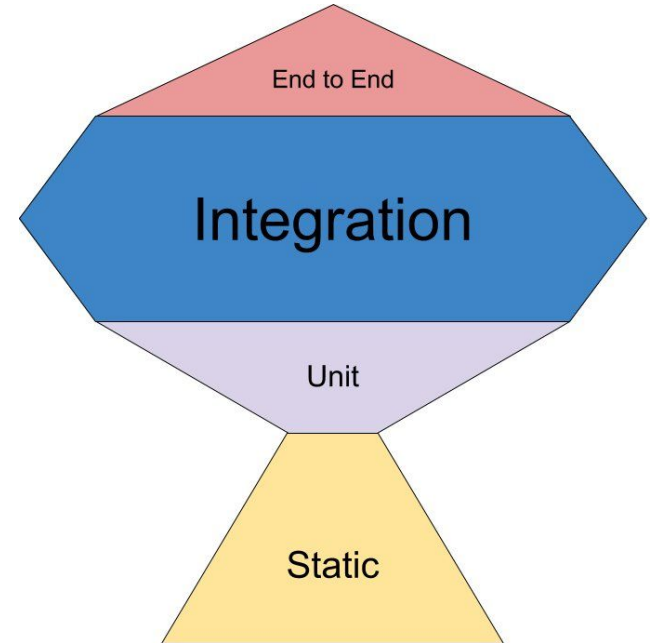
Where can we get to using good patterns?



Testing levels misconception



Source: <https://medium.com/inspiredbrilliance/testing-single-page-applications-b4001adc6452>



Source: <https://x.com/kentcdodds/status/960723172591992832>



”

Frontend app tests should align
with a frontend ecosystem.



What may happen
if we follow books literally?

Add airport

Name

Airport name is required

IATA Code

Country

Regions

- ☐ Aegean Islands
- ☐ Corfu Island (Kerkyra)
- ☐ Ha Long Island
- ☐ Rhodes Island

- ☐ Balearic Islands
- ☒ Crete Island
- ☐ Ionian Islands
- ☐ Tenerife Island

- ☐ Canary Islands
- ☐ Dodecanese Islands
- ☐ Maiorca Island

Vaccination Notes

Submit

#1 Testing too low with redundant mocking

```
// tests/AddAirportView/Name.test.tsx
it('displays message when validation return error' , async () => {
  // Given form hook mocked with error being on
  vi.mocked(useField).mockReturnValue({
    meta: {dirty: true, error: 'Test error'}
  } as FieldRenderProps<string>)

  // When component is rendered with error state
  render(<Name />)

  // Then error message is displayed
  expect(screen.getByText('Test error')).toBeInTheDocument()
})
```

#2 Lack of idea of what should get tested

```
export const AddAirportView = () => {  
  return (<>  
    <FormContainer>  
      <Name />  
      <IataCode />  
      <Country />  
      <Regions />  
      <VaccinationNotes />  
    </FormContainer>  
  </>)  
}
```

#3 Application enhancement example

```
export const AddAirportView = () => {  
  return (<>  
    <Collapsible header={'Important!'}>  
      <p>Important note content</p>  
    </Collapsible>  
    <FormContainer>  
      <Name />  
      <IataCode />  
      <Country />  
      <Regions />  
      <VaccinationNotes />  
    </FormContainer>  
  </>)  
}
```

```
// tests/AddAirportView.test.tsx
it("prints note header and displays content only after click" , async () => {
  // When component render
  render(<AddAirportView />)

  // Then only message header is displayed
  expect(await screen.findByRole('heading', {name: "Important!"}))
    .toBeInTheDocument ()
  expect(screen.queryByText('Important note content' ))
    .not.toBeInTheDocument ()

  // When user click on collapsible element
  await userEvent.click(screen.getByRole('heading', {name: "Important!"}))

  // Then full content is displayed
  expect(await screen.findByText('Important note content' ))
    .toBeInTheDocument ()
})
```

#4 Refactor opportunity (which causes mess on tests)

```
export const AddAirportView = () => {  
  return (<>  
    <ImportantMessage />  
    { /* Component has been moved, yet tests remain in the same file */ }  
    <FormContainer>  
      <Name />  
      <IataCode />  
      <Country />  
      <Regions />  
      <VaccinationNotes />  
    </FormContainer>  
  </>)  
}
```

The background is a solid pink color. In the top right corner, there is a decorative pattern of overlapping geometric shapes, including triangles and squares, in various shades of pink and magenta.

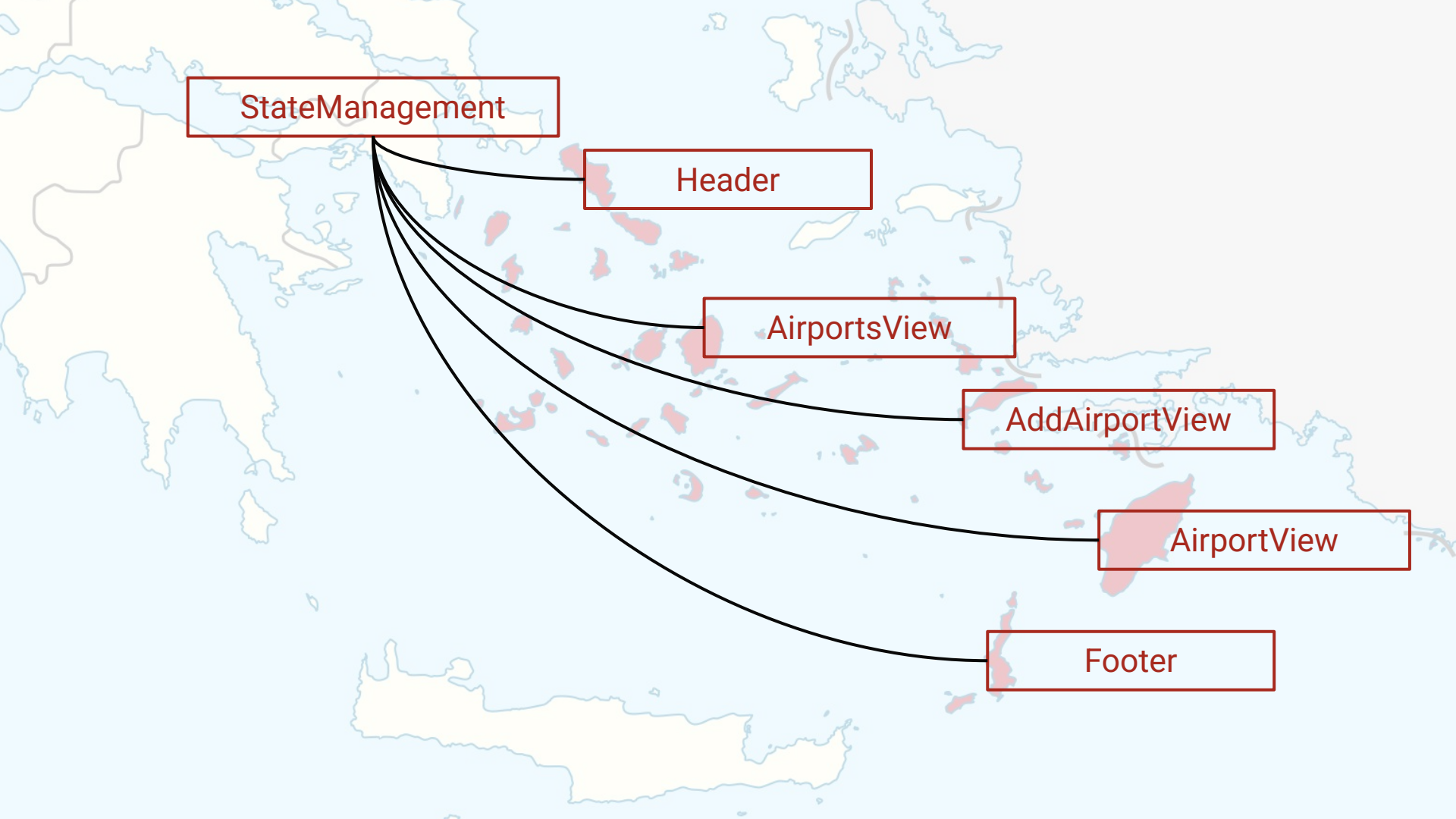
Doing it the right way



Doing it the right way

Island architecture pattern





Island architecture pattern with React Router

```
export const routerDefinition = [
  {
    path: '/', element: <App />, // Ανδρός & Κάρπαθος
    children: [
      {
        path: '/airports',
        children: [
          { index: true, element: <AirportsView />}, // Νάξος
          { path: 'add', element: <AddAirportView />}, // Κως
          { path: ':id', element: <AirportView />}, // Ρόδος
        ],
      },
    ],
  },
] satisfies RouteObject[];
```

First attempt to integration tests

Error: No QueryClient set, use QueryClientProvider to set one

```
› useQueryClient @tanstack/react-query/build/modern/QueryClientProvider.js
› useBaseQuery @tanstack/react-query/build/modern/useBaseQuery.js
› useQuery @tanstack/react-query/build/modern/useQuery.js
› useAirports src/hooks/useAirports.ts
  5| export const useAirports = () => {
  6|   return useQuery({queryFn: getAirports, ...});
  7| };
    | ^
› AddAirportViewGuard src/views/AddAirportView/AddAirportView.guard.tsx
› renderWithHooks react-dom/cjs/react-dom.development.js
...
```

We won't quit dependencies

```
export const App: React.FC = () =>
  <DesignSystemTheme>
    <AppState>
      <ReactQueryContext>
        <AppLayout>
          <Outlet />
          { /* ^ either AirportsView, AirportView or AddAirportView */}
        </AppLayout>
      </ReactQueryContext>
    </AppState>
  </DesignSystemTheme>
```



Doing it the right way

Component wrapper pattern

Component wrapper implementation

```
export const renderWithContexts = (component, options = {}) => {  
  // To include React Router  
  const {path = '/', url = ''} = options;  
  const history = createMemoryHistory({initialEntries: [url]});  
  
  // To include TanStack Query  
  const queryClient = new QueryClient();  
  
  return render(<ReactQueryContext queryClient={queryClient}>  
    <HistoryRouter history={history}>  
      <Routes>  
        <Route path={path} element={component} />  
      </Routes>  
    </HistoryRouter>  
  </ReactQueryContext>)  
};
```

Test of a wrapped component

```
✓ test/integration/Views/AddAirportView.test.tsx (1 test) 1405ms
  ✓ AddAirportView (1)
    ✓ Happy path (1)
      ✓ renders component and adds airport 1402ms
```

```
Test Files 1 passed (1)
  Tests    1 passed (1)
  Start at 10:10:37
  Duration 5.17s
```

Tip #1: Don't bother the performance impact

RUN v3.2.4

Running tests with seed "1757753204226"

Coverage enabled with istanbul

- ✓ test/integration/test-scenarios/AppFooter.test.tsx (2 tests) 31ms
 - ✓ AppFooter (2)
 - ✓ renders raw AppFooter component 11ms
 - ✓ renders AppFooter component wrapped in component wrappers 20ms

Tip #2: Don't render entire App, because of:

- Handling api calls not related with tested view
- Additional navigation to a tested view
- Displaying data being part of other route views
(nasty edge case of nested route views like `'/countries/12/airports/21/airlines'`)
- Complications with stepper user journeys
(go through first $n-1$ steps to test n th step)





Doing it the right way

Integration tests

What is integration tests suite all about?



Kent C. Dodds ⚡️ ✓

@kentcdodds



The more your tests resemble the way your software is used, the more confidence they can give you.

[Przetłumacz wpis](#)

4:05 AM · 23 mar 2018



17



340



1 tys.



65



What frontends integrate with?

- Backend services - api calls
(handled by TanStack Query, mimicked via MSW)
- View display logic - via url (handled by React Router)
- User - data display or interactions via buttons, links, form controls
(handled by HTML/JS, mimicked via Testing Library family)





Doing it the right way

Effective test scenarios

Add airport

Name

Test airport name

IATA Code

TES

Country

Greece ▾

Regions

island

- ☐ Aegean Islands
- ☐ Corfu Island (Kerkyra)
- ☐ Ha Long Island
- ☐ Rhodes Island
- ☐ Balearic Islands
- ☒ Crete Island
- ☐ Ionian Islands
- ☐ Tenerife Island
- ☐ Canary Islands
- ☐ Dodecanese Islands
- ☐ Maiorca Island

Vaccination Notes

Submit

Happy and negative paths approach

```
describe('AddAirportsView', () => {  
  describe('Happy path', () => {  
    // Any scenario leading to functional success:  
    // - all api calls responding 200  
    // - shape of data sent / received is ok  
    // - everything is displayed and work as expected  
  }),  
  describe('Negative path', () => {  
    // Anything stopping us from adding new airport:  
    // - some endpoint responded with error  
    // - user provided invalid data  
  })  
});
```

Test cases for AddAirportView

```
describe('AddAirportsView', () => {  
  describe('Happy path', () => {  
    it('renders view correctly and adds airport' , () => {})  
  })  
  describe('Negative path', () => {  
    it('displays fetch error component on GET api fail' , () => {})  
    it('renders validation error' , () => {})  
    it('displays error message on POST api fail' , () => {})  
  })  
})
```



”

You need only four test cases
to check such a page completely.

Łukasz Nowak, Voxxed Days Crete 2025



Why is this approach
a bullet-proof?

Why is this approach a bullet-proof?

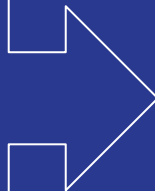
- We know which component should we test and what to assert
 - No more duplicate tests
 - No scenario splitting and moving to other files during source code refactor
 - We check not too much and not too little
- It focuses on functionality instead of implementation details
 - Implementation details does not affect test cases
 - Focusing on integration with an outside world
 - Non-functional changes on the view makes no difference
 - Dependency upgrade or exchange may be transparent for the suite
- The less change tests require, the more assurance they give



Thank you for your attention♡

Q&A

Don't miss the
repository full of
testing examples!



<https://github.com/LukaszNowakPL/bulletproof-frontend-testing>